

Evaluating Amazon EKS for an AWS Infrastructure Decision Utility That Outputs Terraform

Executive summary

Amazon EKS should be added as a supported compute option, but not as the default container recommendation. The strongest case for EKS is not “we run containers,” because AWS already offers lower-overhead ways to do that with Amazon ECS and AWS Fargate. The strongest case is “we need Kubernetes semantics, tooling, portability, or isolation patterns that ECS/Fargate/Lambda do not provide as naturally,” such as Helm charts and operators, DaemonSets, custom networking, soft multi-tenancy, Kubernetes-native platform APIs, or hybrid and on-premises continuity via EKS deployment options. AWS’s own container guidance explicitly frames the first major choice as a serverless operational model versus a Kubernetes operational model, and notes that some organizations invest in Kubernetes because they value the associated skill portability and have teams that can handle frequent cluster upgrades. ¹

For your decision engine, the practical rule is this: **EKS should be selected only when Kubernetes is itself part of the user’s required capability set, or when the user’s platform maturity and long-term portability goals justify the extra cost and operating surface.** Otherwise, the engine should prefer Lambda for event-driven code and ECS or ECS+Fargate for general containerized services. That recommendation is grounded in current AWS service definitions, pricing, and operating guidance: EKS has a per-cluster control-plane fee and a larger day-two operating surface; ECS on EC2 has no additional service fee beyond the AWS resources you run; Fargate charges per vCPU/memory/storage duration; and Lambda charges primarily by requests and compute duration. ²

The most robust product shape is therefore a **two-stage decision**. First, determine whether the user needs Kubernetes at all. Second, if the answer is yes, determine whether the workload should run on EKS managed node groups, EKS Fargate profiles, or a mixed model. Managed node groups should remain the default EKS compute mode for most serious workloads, because EKS Fargate has meaningful constraints around DaemonSets, privileged containers, host networking, alternate CNI plugins, and EBS-backed storage. ³

Scope and assumptions

Several decision-shaping inputs are **unspecified** in your request and should be treated as open variables in the product rules: target customer size, whether the default account topology is single-account or multi-account, whether production use is single-region or multi-region, whether workloads are internet-facing by default, whether customers already have Kubernetes platform staff, and whether “multi-tenancy” means trusted internal tenancy or mutually untrusted external tenants. Those unknowns matter because AWS’s own EKS best-practice guidance treats multi-account structure, cluster access, and tenant-isolation strategy as materially important design decisions rather than implementation details. For example, AWS recommends multiple cluster accounts aligned with the software-development lifecycle at scale, and its

multi-tenant guidance distinguishes soft multi-tenancy inside a cluster from stricter isolation models that are better served by separate clusters. ⁴

This report also assumes a **standard regional EKS deployment model** rather than centering the newer EKS Auto Mode path, because your requested evaluation explicitly asked for managed node groups versus Fargate profiles, OIDC, IRSA or service-account bindings, storage drivers, and add-ons that are most relevant to standard EKS clusters. It also treats “Fargate” carefully: Fargate is a serverless **compute** model for containers, not an orchestrator by itself, and can be used with both ECS and EKS. ⁵

Where EKS belongs in the decision engine

EKS belongs behind a **Kubernetes-need gate**, not behind a generic “containers=yes” gate. AWS defines EKS as a managed Kubernetes service and ECS as a fully managed container orchestration service that simplifies running containers without requiring Kubernetes. AWS also says that one of the first container-service considerations is whether you prefer a serverless operational model or a Kubernetes operational model, and notes that Kubernetes-oriented teams often accept the overhead because they want the skill portability and ecosystem that come with Kubernetes. ⁶

The table below is a concise synthesis of the current service models and pricing structures. It treats **Fargate** as the serverless-container execution model used under ECS or EKS, because that is how AWS documents it. The “portability” and “networking complexity” columns are analytical judgments based on AWS’s definitions of the services and on the Kubernetes-native versus AWS-native control models, not direct quotes. ⁷

Option	Best workload fit	Ops burden	Typical cost profile	Scaling model	Portability	Networking complexity
EKS	Multi-service Kubernetes platforms, operator-heavy stacks, custom networking, soft multi-tenancy, hybrid or portability-driven estates	High	Per-cluster fee plus node or Fargate compute, plus add-ons and team overhead	HPA/VPA plus Cluster Autoscaler or Karpenter, cluster-level scheduling	High relative to ECS	High
ECS	General AWS-native microservices, batch, internal services, container apps without Kubernetes requirements	Low to medium	No extra ECS fee on EC2; or Fargate compute charges if serverless launch type is used	Service/task autoscaling	Lower than EKS	Medium

Option	Best workload fit	Ops burden	Typical cost profile	Scaling model	Portability	Networking complexity
Fargate	Stateless Linux containers where minimizing infrastructure management matters more than deep Kubernetes feature support	Low	Per-vCPU, per-GB, per-second billing; no idle nodes, but higher unit economics than dense EC2 at steady scale	Task/pod autoscaling, right-sized per workload	Medium	Low to medium
Lambda	Event-driven and short-running code, spiky APIs, asynchronous workflows, glue logic	Lowest	Per request and duration; minimal idle-cost floor	Automatic concurrency-based scaling	Lower than container/Kubernetes models	Low

A useful product rule is to give EKS **strong positive weight** when one or more Kubernetes-native requirements are present. Those include: Helm chart or operator dependency; CustomResourceDefinitions that are part of the app platform; DaemonSets; privileged containers; `hostNetwork` or `HostPort`; advanced CNI needs; pod-level security segmentation; soft multi-tenancy through namespaces, RBAC, quotas, and network policies; or hybrid deployment options such as Outposts, Hybrid Nodes, or EKS Anywhere. AWS's docs are explicit on many of these boundaries: EKS is the Kubernetes option; DaemonSets and several other advanced features are unavailable on EKS Fargate; EKS tenant isolation commonly uses namespaces, RBAC, quotas, and network policies; and EKS supports cloud, Outposts, hybrid nodes, and EKS Anywhere deployment options. ⁸

EKS should receive **strong negative weight** when the workload is simple and the team is not prepared to own cluster operations. AWS's container guidance explicitly says some organizations run Kubernetes because they have strong SRE teams and accept frequent upgrades; that implies the inverse as well. If the workload is a small number of stateless services, ordinary HTTP APIs, background workers, or scheduled tasks with no Kubernetes dependency, ECS or Lambda will usually be the better recommendation. That is especially true when team ops maturity is low, budget sensitivity is high, or Kubernetes portability is not a stated goal. ⁹

The most important EKS-specific sub-decision is **managed node groups versus Fargate profiles**. Managed node groups should be the engine's default EKS answer for general-purpose clusters because they automate node provisioning and lifecycle management while still supporting the broad Kubernetes feature surface, including Spot capacity, DaemonSets, alternate CNI approaches, EBS CSI, and the Linux EC2-based pod identity path. By contrast, EKS Fargate is best for Linux-only, stateless, low-ops pods that do not need DaemonSets, privileged containers, `hostNetwork`, `HostPort`, alternate CNI plugins, or EBS-backed

volumes. AWS also notes that EKS Fargate does not support Fargate Spot, and that pods on Fargate run only in private subnets. ¹⁰

A decision utility can translate that into deterministic rules like the following. These are product recommendations derived from the AWS service boundaries above.

```
compute_decision:
  if app_model == event_driven and max_execution_time_minutes <= 15 and not
kubernetes_required:
    choose: lambda

  elif containerized and not kubernetes_required:
    choose: ecs
    launch_model:
      if low_ops_priority == high and not needs_host_network and not
needs_daemonsets and not needs_privileged and not needs_ebs:
        choose: fargate
      else:
        choose: ecs_ec2_or_mixed

  elif kubernetes_required or helm_operators_required or daemonsets_required or
custom_cni_required or soft_multi_tenancy_required or portability_priority ==
high or hybrid_required:
    choose: eks
    eks_compute_mode:
      if daemonsets_required or needs_ebs or needs_gpu or needs_windows or
needs_privileged or needs_host_network or custom_cni_required or spot_tolerance
in [partial, high]:
        choose: managed_node_groups
      elif linux_only and stateless and low_ops_priority == high and not
needs_ebs and not needs_daemonsets and not needs_privileged and not
needs_alt_cni:
        choose: fargate_profiles
      else:
        choose: mixed_managed_nodes_plus_optional_fargate
```

EKS resource model and AWS integrations

A standard EKS resource graph is larger than an ECS service graph because the cluster itself becomes a first-class platform resource. At minimum, the engine must reason about VPC and subnet layout, the EKS control plane, cluster access, node or Fargate compute, pod-to-AWS identity, ingress and load balancing, core add-ons, observability, storage, and image supply chain. AWS's EKS documentation also explicitly connects private-cluster designs to required interface and gateway endpoints for EKS, EKS Auth, STS, ECR, S3, CloudWatch Logs, and other services. ¹¹

The diagram below synthesizes the cluster-to-resource relationships your engine should model for a typical regional EKS deployment with optional managed nodes and optional Fargate profiles. It is derived from AWS EKS user-guide and add-on documentation, plus AWS Load Balancer Controller and storage-driver guidance. ¹²

```

flowchart TD
    U[User intent] --> D[Decision engine]
    D --> V[VPC and subnets]
    D --> C[EKS control plane]
    D --> E[ECR repositories]
    D --> K[KMS keys]
    D --> O[CloudWatch and logs]

    C --> A[Access entries and RBAC]
    C --> I[OIDC provider]
    C --> N[Core add-ons]
    C --> M[Managed node groups]
    C --> F[Fargate profiles]

    M --> P[Pod Identity or IRSA]
    F --> R[IRSA for Fargate workloads]
    M --> S[Cluster Autoscaler or Karpenter]
    M --> T[EBS or EFS or FSx CSI]
    F --> X[EFS static volumes only]

    C --> L[AWS Load Balancer Controller]
    L --> ALB[ALB]
    L --> NLB[NLB]

    V --> EP[VPC endpoints and PrivateLink]
    O --> CW[Container Insights and control plane logs]

```

The resource list below is the practical minimum your decision engine should be able to include or omit deterministically for EKS. The table is a synthesis of AWS EKS cluster, node, Fargate, load-balancer, private-cluster, identity, observability, and storage documentation. ¹³

Layer	Typical AWS resources	When the engine should include them	Notes for Terraform generation
Network foundation	VPC, public/private subnets, route tables, IGW, NAT gateways, security groups, optional VPC endpoints	Always for regional EKS	Private clusters need a larger endpoint set than public clusters

Layer	Typical AWS resources	When the engine should include them	Notes for Terraform generation
EKS control plane	EKS cluster, cluster IAM role, cluster security group, optional KMS encryption config, control-plane log settings	Always	Treat cluster as its own module and state boundary
Access and identity	OIDC provider, access entries, access policies, RBAC manifests, optional external OIDC config	Always	Human access and workload access should be separate concerns
EC2 worker compute	Managed node groups, node IAM roles/instance profiles, launch templates, Auto Scaling groups	Use for most production EKS workloads	Default EKS compute mode
Serverless worker compute	Fargate profiles, pod execution role	Use only for limited Linux-only stateless subsets	Best as a selective adjunct, not the only cluster compute mode in most cases
Image supply chain	ECR repositories, scanning config, lifecycle policies, tag immutability settings	Usually	Especially when the platform also builds or hosts images
Ingress and load balancing	AWS Load Balancer Controller, ALB, NLB, listeners, target groups, ACM certs, Route 53 records	Public or private ingress required	API Gateway can sit in front via VPC Link when API-product features are needed
Autoscaling	Cluster Autoscaler or Karpenter, metrics server, optional HPA/VPA/KEDA plumbing	Node-based clusters with elastic demand	Separate platform policy from application scaling policy
Storage	EBS CSI, EFS CSI, optional FSx CSI, StorageClasses, optional snapshot controller	Stateful Kubernetes workloads	EBS implies EC2 nodes; EFS is the simpler RWX path
Observability	CloudWatch log groups, CloudWatch Observability add-on, Container Insights, Fluent Bit/Fluentd	Production by default	Enable control-plane logs and workload telemetry
Private-cluster connectivity	Interface endpoints for EKS, EKS Auth, STS, ECR API, ECR DKR, ELB, Logs, plus S3 gateway endpoint	Private cluster or no-internet cluster	Required for many common private-cluster workflows

Several caveats are important enough to encode as explicit rules rather than comments. **Managed node groups** automate EC2-node lifecycle and are the correct default when you need Spot nodes, DaemonSets, host networking, privileged containers, alternate CNI approaches, EBS-backed volumes, or broader

Kubernetes compatibility. **Fargate profiles** fit only a narrower subset: Linux-only pods, private subnets only, no DaemonSets, no privileged containers, no `HostPort`, no `HostNetwork`, no alternate CNI plugins, no EBS volumes, and no Fargate Spot. EFS is the main persistent-storage path that remains viable there, and even then Fargate is limited to static provisioning semantics. ¹⁰

For networking, the **default CNI choice should be AWS VPC CNI** for most AWS-native EKS workloads. AWS documents it as the native VPC-networking add-on for EKS, and EKS add-ons install VPC CNI, kube-proxy, and CoreDNS by default. VPC CNI also supports EKS-specific features such as security groups for pods, custom networking, and prefix delegation, although its built-in NetworkPolicy support on EKS is limited to EC2 Linux nodes and does not apply to Fargate or Windows nodes. **Cilium** should be treated as an expert-mode option for EC2-based clusters that need richer eBPF-powered network policy or observability. The Cilium documentation shows a supported chained mode with AWS VPC CNI, but also warns that some advanced Cilium features are limited in chained mode; if those advanced features matter, a fuller move to Cilium may be appropriate. ¹⁴

For storage, your engine should default to **EBS CSI** for zonal block volumes and **EFS CSI** for shared POSIX-style volumes, while treating **Rook/Ceph** as an advanced self-managed alternative rather than an AWS-native default. AWS documents EBS CSI as the driver for EBS-backed Kubernetes volumes, but also states that EBS volumes cannot be mounted to EKS Fargate pods. AWS documents EFS CSI as the EFS path and notes that Fargate supports EFS only with static provisioning. By contrast, Rook positions Ceph as a Kubernetes-native distributed storage platform for block, file, and object storage, but it adds an operator, Ceph daemons, CRDs, and a much larger operational footprint; its own storage architecture guide recommends three or more nodes for a resilient production storage platform. ¹⁵

EKS also integrates broad AWS front-door and data-plane patterns. Use **AWS Load Balancer Controller** for ALB and NLB integration and avoid the legacy cloud provider path that would otherwise create Classic Load Balancers. When API-product features are needed, **API Gateway** can front private EKS services through a **VPC Link** to an ALB or NLB. **CloudFront** can use an ALB, API Gateway, S3, or other custom origins, and **Route 53** alias records are the normal mechanism for attaching your DNS to CloudFront or ELB. For data access, pods typically reach **RDS** over private VPC networking, while **S3** and **DynamoDB** should usually use gateway endpoints instead of hairpinning through NAT. For private EKS clusters, the EKS API itself can be reached with **AWS PrivateLink**, and AWS documents a concrete set of interface and gateway endpoints for common EKS-private-cluster needs. ¹⁶

Security and compliance guardrails

If you add EKS, the decision utility must attach a **mandatory Kubernetes security baseline** that is materially stricter than the baseline you would need for ECS or Lambda. AWS's EKS security guidance is clear that AWS secures the managed control plane, but the customer remains responsible for IAM, pod security, runtime security, network security, and related in-cluster controls. In practice, that means EKS should never be generated without explicit decisions for cluster access, namespace and RBAC design, pod-level hardening, and network isolation. ¹⁷

For **human access**, your default should be **EKS access entries / Cluster Access Management**, not the legacy `aws-auth` ConfigMap. AWS's EKS best-practice guide says ConfigMap-based access management is deprecated and replaced by the Cluster Access Management API, and the user guide says access entries are the best way to grant users access to the Kubernetes API. For **workload access**, AWS now recommends **EKS**

Pod Identity over IRSA where supported, but there is an important caveat: Pod Identity currently applies only to Linux EC2 worker nodes on EKS and is not available on Outposts, EKS Anywhere, or Fargate. That means your engine should prefer Pod Identity for EC2-backed clusters, but automatically fall back to **IRSA** for EKS Fargate and for certain hybrid patterns. ¹⁸

For **pod and namespace security**, enforce at least Kubernetes **Pod Security Standards Baseline** for ordinary application namespaces and strongly consider **Restricted** for regulated namespaces. Kubernetes defines three Pod Security profiles — Privileged, Baseline, and Restricted — with Baseline preventing known privilege escalations and Restricted following stronger hardening practices. RBAC should be least-privilege and namespace-scoped by default, with cluster-admin limited to a tightly controlled break-glass path. For multi-tenant shared clusters, also require namespaces, roles and role bindings, resource quotas, limit ranges, and network policies; that is exactly how AWS describes “soft multi-tenancy” on EKS. If the tenancy boundary is hard or mutually untrusted, your engine should escalate to separate clusters or accounts rather than trying to solve the problem purely with in-cluster policy. ¹⁹

For **network security**, require a default-deny NetworkPolicy posture for application namespaces and pair it with EKS-specific controls where appropriate. Kubernetes notes that pods are non-isolated by default until a NetworkPolicy applies, and the AWS VPC CNI network-policy feature on EKS works only on EC2 Linux nodes. If compliance or tenant isolation requires AWS-native segmentation to external services, use **security groups for pods**. If the cluster must be fully private, require **private endpoint access** for the cluster API and the documented set of VPC endpoints for EKS, EKS Auth, STS, ECR, S3, ELB, and CloudWatch Logs; AWS explicitly says private-cluster nodes require private endpoint access and documents the endpoint matrix needed for common workloads. ²⁰

For **encryption and auditability**, always enable EKS control-plane logs, and for production or regulated workloads turn on all documented control-plane log types (`api` , `audit` , `authenticator` , `controllerManager` , and `scheduler`). On current EKS versions, AWS provides default envelope encryption for all Kubernetes API data on clusters running Kubernetes 1.28 or higher, but that does **not** cover node disks or EBS volumes, so your engine should still model KMS and storage-level encryption separately. AWS also supplies an AWS Config rule that checks whether EKS clusters encrypt secrets with KMS. For workload telemetry, add the CloudWatch Observability add-on or otherwise install the CloudWatch and logging agents required for Container Insights. ²¹

For the **image and artifact chain**, ECR repositories should default to private, with scan-on-push or registry-level scanning, lifecycle policies, and tag-management rules. AWS states that ECR supports vulnerability scanning and lifecycle policies, and the repository-creation flow also supports tag immutability and KMS-backed encryption options. Those controls should be paired with admission or CI policy in higher-maturity environments, but even in an MVP decision engine they should be surfaced as default posture choices rather than optional afterthoughts. ²²

Hybrid variants change the rules and should be surfaced as explicit constraints. AWS documents EKS on Outposts as either extended clusters or local clusters, and it documents EKS Anywhere as the self-contained air-gapped option. At the same time, AWS says IRSA is not available for local clusters on Outposts, and Pod Identity is not available on Outposts or EKS Anywhere. That means your engine cannot treat hybrid EKS variants as simple parameter changes to a standard regional-cluster template; it must branch identity and sometimes networking assumptions as well. ²³

Cost and operational tradeoffs

The biggest economic trap in EKS evaluations is pretending that “managed Kubernetes” is priced like “managed containers.” It is not. **Amazon EKS charges per cluster per hour**, currently **\$0.10 per cluster hour** for standard Kubernetes version support and **\$0.60 per cluster hour** during extended support. That is before you pay for EC2 worker nodes, Fargate pods, storage, data transfer, or observability. AWS also now offers **Provisioned Control Plane** tiers for especially large or bursty clusters at additional hourly rates. By contrast, **Amazon ECS on EC2 has no additional ECS service fee**, **Fargate charges per-second for vCPU, memory, and extra storage**, and **Lambda charges by requests and compute duration**. ²⁴

The chart below is therefore **qualitative**, not a price quote. It combines three factors: fixed platform-cost floor, feature-related engineering overhead, and the amount of utilization you typically need before denser EC2 packing starts to outperform serverless-unit billing. The underlying service-pricing and operating-model facts come from AWS pricing pages and EKS best-practice guidance; the 1–5 scores themselves are an analytical synthesis for product-design purposes. ²⁵

```
xychart-beta
  title "Relative fixed-cost floor and platform overhead"
  x-axis ["Lambda", "ECS/Fargate", "ECS/EC2", "EKS/Fargate", "EKS/EC2"]
  y-axis "Relative score" 0 --> 5
  bar [1, 2, 2, 3, 5]
```

Operationally, EKS’s extra cost is not only the cluster fee. It is also the work of staying current with Kubernetes and cluster add-ons. AWS’s container guide explicitly notes that Kubernetes-oriented teams handle frequent cluster upgrades, and current EKS version guidance says a minor version stays under standard support for the first 14 months and then enters a 12-month extended-support window at higher cost. If your users do not already have a platform team or a strong appetite for Kubernetes operations, your engine should bias away from EKS even if the workload is containerized. ²⁶

Within EKS itself, there are sharp tradeoffs between **managed node groups** and **Fargate profiles**. Managed node groups reduce EC2 node-lifecycle work and support Spot rebalancing, making them the best default for most real-world clusters. AWS recommends Spot capacity for stateless, interruption-tolerant workloads and On-Demand for fault-intolerant and stateful workloads. EKS-managed Spot node groups can automatically attempt replacement and drain affected nodes when rebalance recommendations arrive, which gives your engine a clean way to model “cheap but interruption-tolerant” versus “stable and stateful.” ²⁷

Fargate for EKS pods reduces node operations and bills per second, but it does **not** remove the EKS cluster fee and it sacrifices a lot of cluster feature surface. AWS positions Fargate as right-sized on-demand compute that eliminates node-group sizing and packing work, which is economically attractive for small numbers of isolated stateless pods or unpredictable platform-sidecars. It becomes less attractive when you need broad Kubernetes compatibility, high cluster density, or deep scheduling optimization. ²⁸

Cluster density and right-sizing materially affect EKS economics. AWS’s EKS best-practice guidance explains that overly large pod requests reduce packing efficiency and that right-sizing workloads, then combining that with autoscaling, is how you reduce total spend. AWS also notes that PodDisruptionBudgets that are

too restrictive can block scale-down for Cluster Autoscaler and Karpenter. That means your decision utility should not only emit infrastructure; it should also emit **operational warnings** when the selected EKS path is likely to be cost-inefficient without good resource requests, disruption budgets, and autoscaling policy.

29

For node autoscaling, AWS's current guidance covers both **Cluster Autoscaler** and **Karpenter**. Cluster Autoscaler scales predefined node groups and respects ASG minimum and maximum settings; Karpenter provisions right-sized EC2 resources just in time and reduces the need to predefine many node groups. If your product is choosing an EKS architecture in 2026, it should treat autoscaler choice as part of the architecture decision, not as an implementation footnote. A reasonable default is: use **Karpenter** for heterogeneous EC2-backed clusters where cost and flexibility matter, and use **Cluster Autoscaler** when the organization already standardizes on ASG-backed managed node groups or wants a more conservative pattern.

30

Terraform generation contract

The cleanest way to support EKS in your product is to make Terraform the **final rendering** of an EKS architecture contract, not the thing that decides architecture. HashiCorp's module guidance recommends composable, reusable modules; scoping by encapsulation, privilege boundaries, and volatility; and keeping provider configuration in the root module. HashiCorp also says each child module should declare provider requirements even though provider configurations themselves are global, and AWS prescriptive guidance recommends Amazon S3 as the default remote backend foundation for Terraform on AWS. Current Terraform S3 backend documentation supports `use_lockfile = true`, and HashiCorp recommends committing `.terraform.lock.hcl` so provider selections are reproducible and reviewable.

31

That implies four generation rules for EKS. First, separate **low-volatility, high-privilege platform modules** such as VPC and cluster control plane from **faster-moving add-on and workload modules**. Second, keep provider definitions in the root and pass providers into child modules as needed. Third, pin provider versions through `required_providers`, use a root-level dependency lock file, and upgrade providers deliberately. Fourth, split Terraform state by privilege and lifecycle — for example `network`, `eks-platform`, `eks-addons`, and `workloads` — rather than blending everything into one state file. Those choices directly reflect HashiCorp's privilege-and-volatility guidance and AWS's backend recommendations.

32

The EKS-specific user-input extension below captures the minimum inputs that materially change the resource graph. The field choices are based on AWS's documented EKS feature boundaries, especially around compute mode, CNI, access model, tenancy, storage, and private-cluster operation.

33

```
eks_extension:
  kubernetes_required: boolean
  kubernetes_dependency_reason:
    - helm_charts
    - operators_or_crds
    - daemonsets
    - privileged_or_host_network
    - custom_cni
```

- soft_multi_tenancy
- portability_or_hybrid
- existing_kubernetes_standard

cluster_scope:

environments: [dev, staging, prod]
 account_topology: [unspecified, single_account, multi_account]
 region_strategy: [unspecified, single_region, multi_region]

compute:

eks_mode: [managed_node_groups, fargate_profiles, mixed]
 linux_only: boolean
 windows_required: boolean
 gpu_required: boolean
 spot_tolerance: [none, partial, high]

networking:

endpoint_mode: [public_limited, private_only, dual]
 no_outbound_internet: boolean
 cni_choice: [aws_vpc_cni, cilium_chained, cilium_exclusive]
 security_groups_for_pods: boolean
 custom_networking: boolean

tenancy:

isolation_model: [single_tenant, soft_multi_tenant, hard_multi_tenant]
 tenant_trust_level: [trusted, mixed, untrusted]

storage:

persistent_storage_mode: [none, ebs_rwo, efs_rwx, fsx, ceph_rook]
 volume_encryption_kms: boolean

identity_and_access:

human_access_model: [access_entries_cam, external_oidc, unspecified]
 workload_access_model: [pod_identity, irsa, mixed]
 secrets_source: [k8s_secret, secrets_manager_csi, parameter_store_csi]

ingress:

north_south_entry: [alb, nlb, api_gateway_vpc_link, cloudfront_alb, cloudfront_api]
 route53_managed: boolean

observability:

control_plane_logs: [disabled, essential, all]
 container_insights: boolean

The intermediate spec should then normalize the user's intent into a resource-and-policy contract that is stable enough to generate Terraform deterministically.

```

{
  "platform": "eks",
  "topology": {
    "account_topology": "unspecified",
    "endpoint_mode": "private_only",
    "internet_egress": "via_vpc_endpoints_and_limited_nat"
  },
  "compute": {
    "mode": "mixed",
    "managed_node_groups": [
      {
        "name": "system",
        "capacity_type": "ON_DEMAND",
        "instance_families": ["m7g", "m7i"],
        "min": 2,
        "max": 6
      },
      {
        "name": "apps-spot",
        "capacity_type": "SPOT",
        "instance_families": ["c7g", "m7g", "c7i"],
        "min": 0,
        "max": 20
      }
    ],
    "fargate_profiles": [
      {
        "name": "serverless-jobs",
        "namespaces": ["jobs"]
      }
    ]
  },
  "networking": {
    "cni": "aws_vpc_cni",
    "security_groups_for_pods": true,
    "private_endpoints": ["eks", "eks-auth", "sts", "ecr.api", "ecr.dkr",
"logs", "elasticloadbalancing"],
    "gateway_endpoints": ["s3", "dynamodb"]
  },
  "identity": {
    "human_access": "access_entries_cam",
    "workload_access": "mixed",
    "oidc_provider": true
  },
  "ingress": {
    "controller": "aws_load_balancer_controller",
    "entry": "api_gateway_vpc_link",

```

```

    "internal_service_lb": "alb"
  },
  "storage": {
    "drivers": ["aws-ebs-csi-driver", "aws-efs-csi-driver", "snapshot-
controller"],
    "default_app_storage": "ebs_rwo"
  },
  "security": {
    "pod_security_standard": "restricted",
    "control_plane_logs": ["api", "audit", "authenticator",
"controllerManager", "scheduler"],
    "image_scanning": true,
    "ecr_lifecycle_policy": true
  },
  "observability": {
    "cloudwatch_observability_addon": true,
    "container_insights": true
  },
  "generation_status": {
    "ready_for_terraform": true,
    "unresolved_questions": []
  }
}

```

The module mapping below is the recommended Terraform composition pattern. It follows HashiCorp's advice to scope modules by privilege and volatility, keeps provider configs at the root, and separates cluster platform state from in-cluster objects that depend on the cluster already existing. ³⁴

Module	Primary inputs	Primary outputs	Recommended state boundary
network_foundation	CIDRs, AZ count, public/private subnet strategy, NAT policy, endpoint policy	vpc_id , private_subnet_ids , public_subnet_ids , endpoint_sg_id	Yes
kms_foundation	Key policy mode, rotation, aliases, encryption consumers	kms_key_arns	Usually yes
ecr_repositories	Repo names, immutability, scan mode, lifecycle rules	Repository URLs, repo ARNs	Optional separate

Module	Primary inputs	Primary outputs	Recommended state boundary
<code>eks_control_plane</code>	Cluster name, version, VPC/subnets, endpoint mode, log types, encryption config	Cluster ARN, endpoint, CA data, OIDC issuer URL, cluster SG	Yes
<code>eks_access</code>	Access-entry definitions, access policies, RBAC principals	Access-entry IDs, RBAC manifests or rendered objects	Often separate from cluster
<code>eks_node_groups</code>	Node-group definitions, launch templates, Spot or On-Demand mix	Node role ARNs, node-group names, ASG names	Yes
<code>eks_fargate_profiles</code>	Namespace selectors, pod execution role settings	Fargate profile names, execution role ARN	Same or separate from node groups
<code>eks_addons_core</code>	Add-on versions and configs for VPC CNI, CoreDNS, kube-proxy, Pod Identity agent	Add-on IDs/status	Yes
<code>eks_ingress</code>	ALB Controller config, IAM policy ARN, ACM, ingress mode	Controller SA/IAM role, LB ARNs/DNS	Separate from cluster
<code>eks_autoscaling</code>	Autoscaler mode, ASG/nodepool policy, metrics inputs	Autoscaler release IDs, IAM role ARNs	Separate from cluster
<code>eks_storage</code>	CSI driver choices, StorageClasses, encryption defaults	StorageClass names, IAM role ARNs	Separate from cluster
<code>eks_observability</code>	Control-plane log groups, CloudWatch add-on, Fluent Bit settings	Log group names, add-on IDs	Separate from cluster

Module	Primary inputs	Primary outputs	Recommended state boundary
<code>eks_workload_identity</code>	Pod Identity associations and/or IRSA bindings per service account	Association IDs, role ARNs	Separate from platform base

For the root module, use an S3 backend with state locking and provider pinning. HashiCorp documents `use_lockfile = true` for S3 backends and recommends version constraints plus a committed dependency lock file. AWS recommends S3 for remote state on AWS. ³⁵

```

terraform {
  required_version = "~> 1.14.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.0"
    }
    helm = {
      source  = "hashicorp/helm"
      version = "~> 3.0"
    }
  }

  backend "s3" {
    bucket     = "org-tf-state"
    key        = "eks-platform/prod/terraform.tfstate"
    region     = "us-east-1"
    use_lockfile = true
  }
}

```

A sample module-manifest list for your generator could look like this. This is an example composition pattern, not a claim that these names are mandated by AWS or HashiCorp.

```

terraform_manifest:
  root:
    providers: [aws, kubernetes, helm]
    states:

```

- name: network
key: network/prod.tfstate
- name: eks_platform
key: eks-platform/prod.tfstate
- name: eks_addons
key: eks-addons/prod.tfstate
- name: workloads
key: workloads/prod.tfstate

calls:

- module: network_foundation
state: network
- module: kms_foundation
state: eks_platform
- module: ecr_repositories
state: eks_platform
- module: eks_control_plane
state: eks_platform
depends_on: [network_foundation, kms_foundation]
- module: eks_access
state: eks_platform
depends_on: [eks_control_plane]
- module: eks_node_groups
state: eks_platform
depends_on: [eks_control_plane]
- module: eks_fargate_profiles
state: eks_platform
depends_on: [eks_control_plane]
- module: eks_addons_core
state: eks_addons
depends_on: [eks_control_plane, eks_node_groups]
- module: eks_ingress
state: eks_addons
depends_on: [eks_addons_core]
- module: eks_autoscaling
state: eks_addons
depends_on: [eks_node_groups]
- module: eks_storage

```

state: eks_addons
depends_on: [eks_addons_core]

- module: eks_observability
  state: eks_addons
  depends_on: [eks_addons_core]

- module: eks_workload_identity
  state: workloads
  depends_on: [eks_addons_core]

```

The final product recommendation is therefore straightforward: **support EKS, but only as a scored architecture branch that activates when Kubernetes-specific requirements are explicit or strongly implied.** In the generated decision rules, make **managed node groups the default EKS compute mode**, allow **Fargate profiles only for clearly compatible subsets**, default the network to **AWS VPC CNI**, surface **Cilium only as an advanced EC2-only option**, and treat **cluster access, workload identity, private endpoints, storage drivers, and ingress controllers as first-class platform decisions** rather than late Terraform details. That product shape is the one most aligned with current AWS and HashiCorp guidance, and it is the one most likely to preserve the real value of your utility: turning end-user intent into an architecture decision, with Terraform as the output rather than the entire product. ³⁶

¹ ⁵ ⁶ ⁷ ⁸ ⁹ ²⁶ ³⁶ <https://docs.aws.amazon.com/decision-guides/latest/containers-on-aws-how-to-choose/choosing-aws-container-service.html>

<https://docs.aws.amazon.com/decision-guides/latest/containers-on-aws-how-to-choose/choosing-aws-container-service.html>

² ²⁴ ²⁵ <https://aws.amazon.com/eks/pricing/>

<https://aws.amazon.com/eks/pricing/>

³ ¹⁰ ²⁷ ³³ <https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html>

<https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html>

⁴ <https://docs.aws.amazon.com/eks/latest/best-practices/multi-account-strategy.html>

<https://docs.aws.amazon.com/eks/latest/best-practices/multi-account-strategy.html>

¹¹ ¹³ <https://aws.amazon.com/eks/features/>

<https://aws.amazon.com/eks/features/>

¹² <https://docs.aws.amazon.com/eks/latest/userguide/eks-add-ons.html>

<https://docs.aws.amazon.com/eks/latest/userguide/eks-add-ons.html>

¹⁴ <https://docs.aws.amazon.com/eks/latest/userguide/workloads-add-ons-available-eks.html>

<https://docs.aws.amazon.com/eks/latest/userguide/workloads-add-ons-available-eks.html>

¹⁵ <https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html>

<https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html>

¹⁶ <https://docs.aws.amazon.com/eks/latest/userguide/aws-load-balancer-controller.html>

<https://docs.aws.amazon.com/eks/latest/userguide/aws-load-balancer-controller.html>

¹⁷ <https://docs.aws.amazon.com/eks/latest/best-practices/security.html>

<https://docs.aws.amazon.com/eks/latest/best-practices/security.html>

- 18 <https://docs.aws.amazon.com/eks/latest/best-practices/cluster-access-management.html>
<https://docs.aws.amazon.com/eks/latest/best-practices/cluster-access-management.html>
- 19 <https://kubernetes.io/docs/concepts/security/pod-security-standards/>
<https://kubernetes.io/docs/concepts/security/pod-security-standards/>
- 20 <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
<https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- 21 <https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html>
<https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html>
- 22 <https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html>
<https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html>
- 23 <https://docs.aws.amazon.com/eks/latest/userguide/eks-outposts.html>
<https://docs.aws.amazon.com/eks/latest/userguide/eks-outposts.html>
- 28 <https://docs.aws.amazon.com/eks/latest/userguide/fargate.html>
<https://docs.aws.amazon.com/eks/latest/userguide/fargate.html>
- 29 https://docs.aws.amazon.com/eks/latest/best-practices/node_and_workload_efficiency.html
https://docs.aws.amazon.com/eks/latest/best-practices/node_and_workload_efficiency.html
- 30 <https://docs.aws.amazon.com/eks/latest/userguide/autoscaling.html>
<https://docs.aws.amazon.com/eks/latest/userguide/autoscaling.html>
- 31 32 34 <https://developer.hashicorp.com/terraform/tutorials/modules/pattern-module-creation>
<https://developer.hashicorp.com/terraform/tutorials/modules/pattern-module-creation>
- 35 <https://developer.hashicorp.com/terraform/language/backend/s3>
<https://developer.hashicorp.com/terraform/language/backend/s3>